

Tomato Leaf Disease Classification Using CNN

Abstract

Plant diseases pose a significant threat to agricultural productivity and food security worldwide. Early and accurate detection of plant diseases is crucial for effective crop management. In this project, a deep learning-based approach using Convolutional Neural Networks (CNN) was developed for the classification of tomato leaf diseases. A custom CNN architecture was built from scratch to classify tomato leaf images into **10 different categories**, including 9 disease classes and 1 healthy class. The model was trained on a publicly available dataset containing **10,000 training images** and 1,000 validation images. The final model achieved approximately **94%** training accuracy with a validation accuracy of **88.8%**, demonstrating the effectiveness of CNNs for plant **disease classification tasks**.

Table of Contents

1. Introduction
 2. Problem Statement
 3. Objectives
 4. Dataset Description
 5. Data Preprocessing
 6. Model Architecture
 7. Training Process
 8. Model Evaluation
 9. Results
 10. Future Improvements
 11. Conclusion
 12. References
-

1. Introduction

Plant diseases are a major concern in agriculture, leading to significant crop losses and reduced food production. Traditional methods of disease detection rely on visual inspection by experts, which is time-consuming, subjective, and often impractical for large-scale farming operations.

The advent of deep learning, particularly Convolutional Neural Networks (CNNs), has revolutionized the field of image classification. CNNs have demonstrated remarkable performance in various computer vision tasks, including plant disease detection. By learning hierarchical features directly from raw pixel data, CNNs can automatically identify disease symptoms with high accuracy.

This project aims to develop a CNN-based classifier for tomato leaf diseases. Tomatoes are one of the most widely cultivated vegetables globally, and they are susceptible to numerous diseases that can significantly impact yield and quality. Early and accurate detection of these diseases can enable timely intervention, reducing crop losses and improving agricultural productivity.

2. Problem Statement

Develop a multi-class image classification model that can accurately classify tomato leaf images into one of the following 10 categories:

1. Septoria Leaf Spot
2. Target Spot
3. Bacterial Spot
4. Early Blight
5. Late Blight
6. Leaf Mold
7. Healthy
8. Tomato Mosaic Virus
9. Tomato Yellow Leaf Curl Virus
10. Spider Mites (Two-spotted Spider Mite)

The model should generalize well to unseen images while maintaining high classification accuracy across all disease categories.

3. Objectives

The primary objectives of this project are:

- Build a CNN-based multi-class image classifier.
 - Preprocess and augment the dataset for optimal training.
 - Design and train a CNN architecture from scratch.
 - Implement proper regularization techniques to prevent overfitting.
 - Evaluate model performance using multiple evaluation metrics.
 - Achieve high classification accuracy on both training and validation datasets.
-

4. Dataset

Dataset Name

Tomato Leaf Disease Dataset

KAUSTUBH B · UPDATED 6 YEARS AGO

▲ 342

<> Code

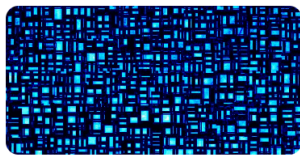
Download

🌟

⋮

Tomato leaf disease detection

Tomato leaf disease detection using CNN



Data Card

Code (223)

Discussion (1)

Suggestions (0)

About Dataset

The data has different types of diseases for tomato leaves.
Here goes the list:

- Tomato_mosaic_virus
- Target_Spot
- Bacterial_spot
- Tomato_Yellow_Leaf_Curl_Virus

Usability ⓘ
6.88

License
[CC0: Public Domain](#)

Expected update frequency
Not specified

Tags

Kaggle Dataset

[kaustubhb999/tomatoleaf](https://www.kaggle.com/kaustubhb999/tomatoleaf)

Tomato leaf disease detection

342

Code

Download

Data Card

Code (223)

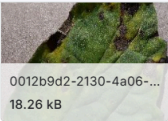
Discussion (1)

Suggestions (0)

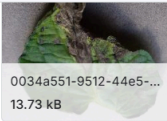
← Tomato__Early_blight (1000 files)

About this directory

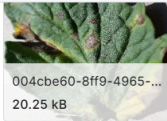
This directory does not have a description yet.



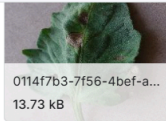
0012b9d2-2130-4a06-...
18.26 kB



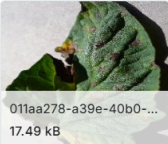
0034a551-9512-44e5-...
13.73 kB



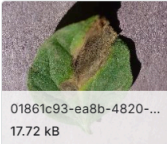
004cbe60-8ff9-4965-...
20.25 kB



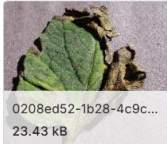
0114f7b3-7f56-4bef-a...
13.73 kB



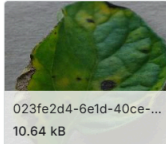
011aa278-a39e-40b0-...
17.49 kB



01861c93-ea8b-4820-...
17.72 kB



0208ed52-1b28-4c9c...
23.43 kB



023fe2d4-6e1d-40ce-...
10.64 kB

Suggest Edits

Data Explorer

186.22 MB

tomato

train

Tomato__Bacterial_

Tomato__Early_blig

0012b9d2-2130-...

0034a551-9512-...

004cbe60-8ff9-...

0114f7b3-7f56-4...

011aa278-a39e-...

01861c93-ea8b-...

0208ed52-1b28-...

023fe2d4-6e1d-...

0378db62-5231-...

0401adc0-0998-...

04747279-48b2-...

04ddbde3-d33c-...

04e38351-80bc-...

Dataset Distribution

The dataset contains images of tomato leaves belonging to 10 different categories:

Class	Training Images	Validation Images
Septoria Leaf Spot	1000	100
Target Spot	1000	100
Bacterial Spot	1000	100
Early Blight	1000	100

Late Blight	1000	100
Leaf Mold	1000	100
Healthy	1000	100
Tomato Mosaic Virus	1000	100
Tomato Yellow Leaf Curl Virus	1000	100
Spider Mites	1000	100

Total Training Images: 10,000

Total Validation Images: 1,000

The dataset is well-balanced with exactly 1000 images per class in the training set and 100 images per class in the validation set.

5. Data Preprocessing

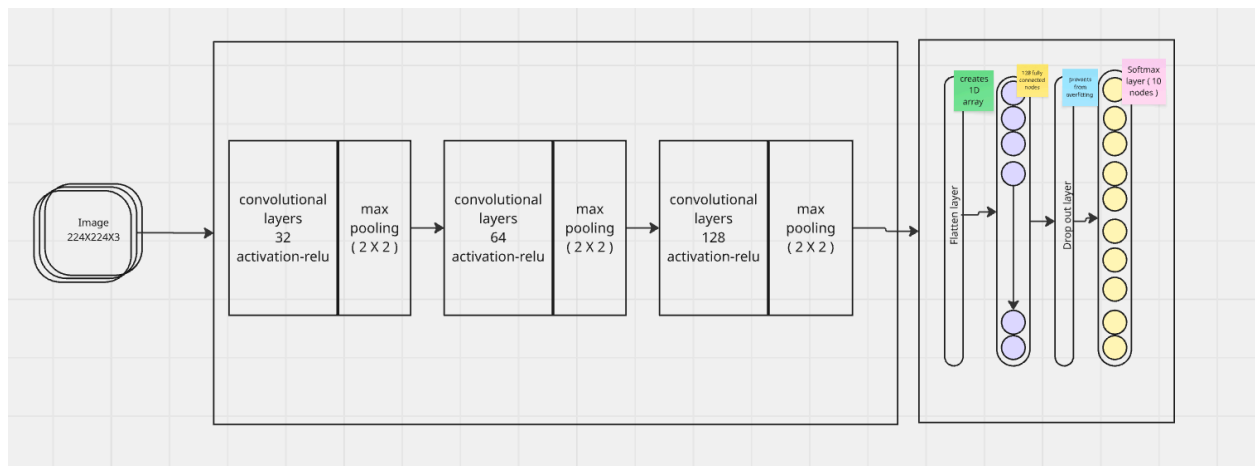
The following preprocessing techniques were applied before training:

1. **Image Resizing:** All images were resized to 224×224 pixels to ensure consistent input dimensions for the CNN.
2. **Array Conversion:** Images were converted into NumPy arrays for efficient processing.
3. **Label Encoding:** Categorical labels were encoded into numerical values using `LabelEncoder`.
4. **Data Shuffling:** The dataset was randomly shuffled using `sklearn.utils.shuffle` to ensure an unbiased distribution of samples during training.

The preprocessing ensured that every image had the same input dimensions and the data was properly formatted for model training.

6. Model Architecture

A CNN was built from scratch using TensorFlow/Keras. The architecture consists of the following layers:



Architecture Overview

1. Input Layer: $224 \times 224 \times 3$ (RGB images)
2. Rescaling Layer: Normalizes pixel values to $[0, 1]$
3. Convolutional Block 1:
 - Conv2D: 32 filters, 3×3 kernel, ReLU activation
 - MaxPooling2D: 2×2 pool size
4. Convolutional Block 2:
 - Conv2D: 64 filters, 3×3 kernel, ReLU activation
 - MaxPooling2D: 2×2 pool size
5. Convolutional Block 3:
 - Conv2D: 128 filters, 3×3 kernel, ReLU activation
 - MaxPooling2D: 2×2 pool size
6. Flatten Layer: Flattens the 3D feature maps
7. Fully Connected Layer: 128 neurons, ReLU activation
8. Dropout Layer: 50% dropout rate (regularization)

9. Output Layer: 10 neurons, Softmax activation (multi-class classification)

Model Summary

Layer (type)	Output Shape	Param #
rescaling	(None, 224, 224, 3)	0
conv2d_42 (Conv2D)	(None, 222, 222, 32)	896
max_pooling2d_42	(None, 111, 111, 32)	0
conv2d_43 (Conv2D)	(None, 109, 109, 64)	18,496
max_pooling2d_43	(None, 54, 54, 64)	0
conv2d_44 (Conv2D)	(None, 52, 52, 128)	73,856
max_pooling2d_44	(None, 26, 26, 128)	0
flatten_14	(None, 86528)	0
dense_35 (Dense)	(None, 128)	11,075,712
dropout_21	(None, 128)	0
dense_36 (Dense)	(None, 10)	1,290

Total Parameters: 11,170,250 (42.61 MB)

Trainable Parameters: 11,170,250 (42.61 MB)

Non-trainable Parameters: 0 (0.00 B)

7. Training Process

The model was trained using the following configuration:

Parameter	Value
Optimizer	Adam
Loss Function	Sparse Categorical Crossentropy
Metric	Accuracy
Batch Size	32
Epochs	15
Input Size	$224 \times 224 \times 3$

The training process involved:

1. Data Split: Training data (10,000 images) and validation data (1,000 images)
 2. Data Shuffling: Both training and validation sets were shuffled before training
 3. Training: The model was trained for 15 epochs with batch size 32
 4. Validation: Validation accuracy and loss were monitored after each epoch
-

8. Model Evaluation

The model performance was evaluated using:

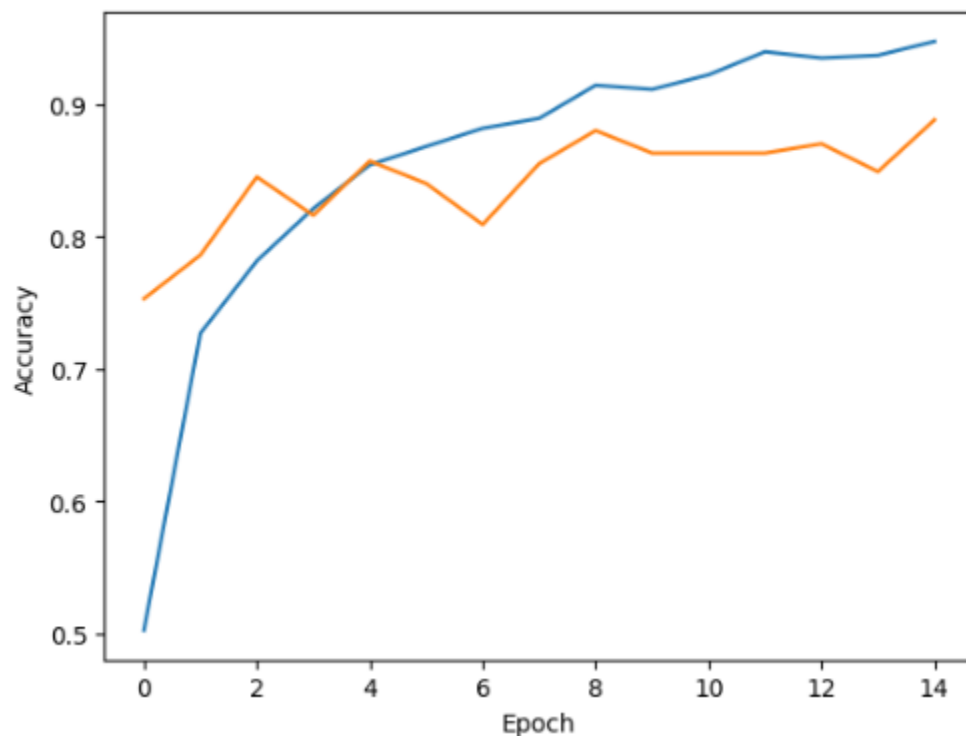
- Accuracy: Training and validation accuracy
- Loss: Training and validation loss
- ROC Curve: Receiver Operating Characteristic curve and Area Under Curve (AUC) for multi-class classification

These metrics provide insight into both the learning process and the model's ability to generalize to unseen data.

Training Accuracy

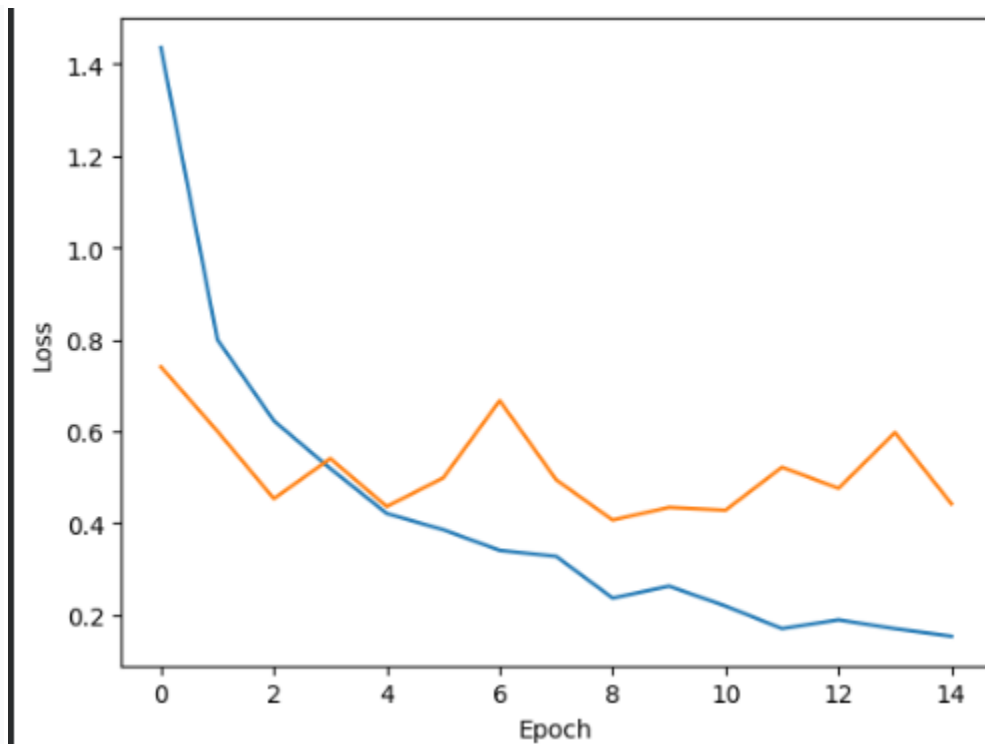
The training accuracy steadily increased throughout the epochs, reaching approximately 94% by the end of training. The validation accuracy showed a similar upward trend, indicating that the model learned meaningful features without severe overfitting.

Final Validation Accuracy: 88.8%



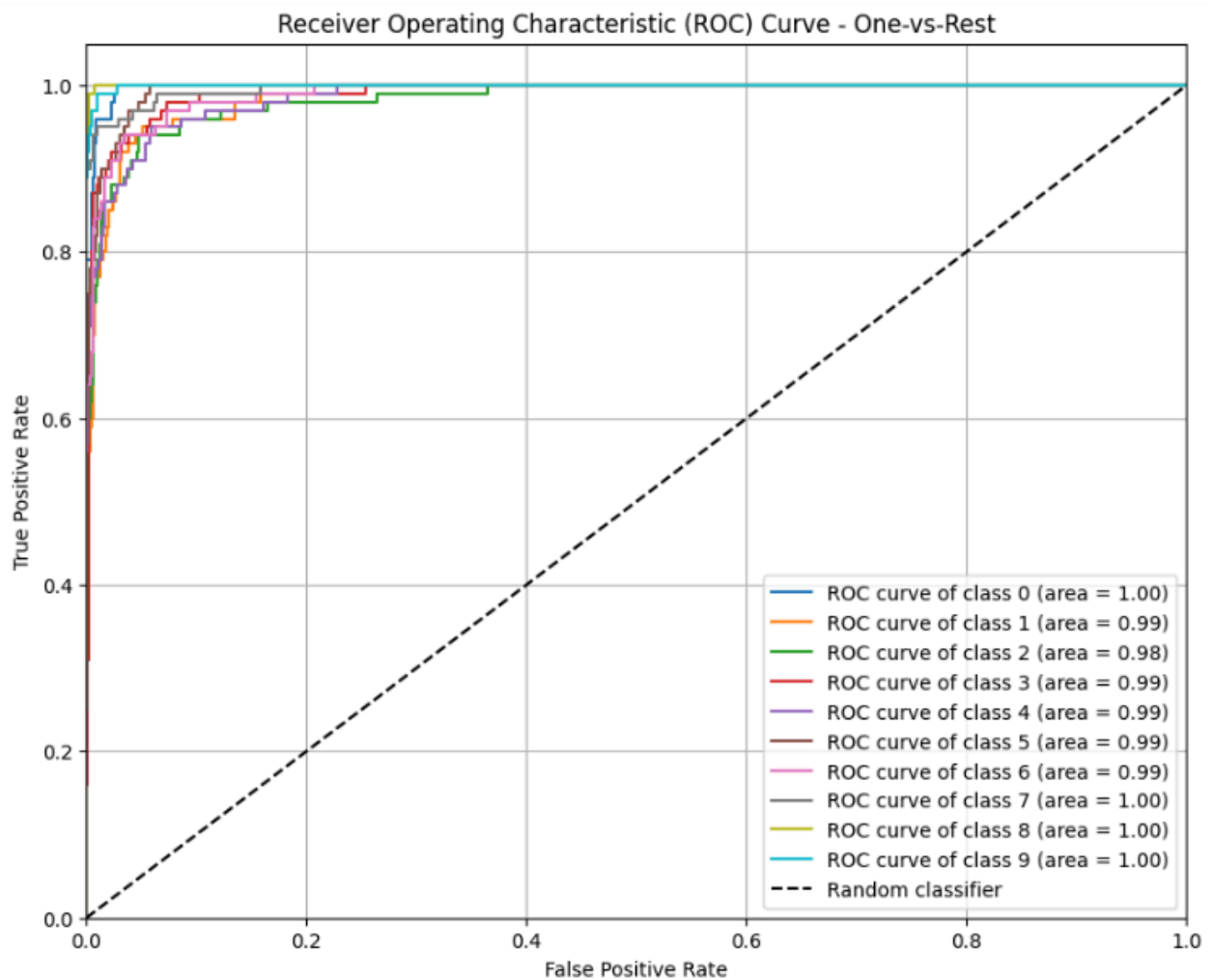
Training Loss

The training loss decreased consistently with each epoch, demonstrating effective learning. The validation loss also showed a general downward trend, indicating that the model generalized well to unseen data.



ROC Curve and AUC

The Receiver Operating Characteristic (ROC) curve measures the model's ability to distinguish between classes across all 10 categories. The Area Under Curve (AUC) scores for each class were excellent, ranging from 0.98 to 1.00, indicating outstanding classification capability.



9. Results

Metric	Value
Final Training Accuracy	~94.7%
Final Validation Accuracy	~88.8%
Final Training Loss	~0.154
Final Validation Loss	~0.442
Dataset Size	10,000 Training, 1,000 Validation
Number of Classes	10

The model demonstrated strong performance with excellent discrimination across all 10 disease categories. The high ROC-AUC scores indicate that the model can effectively distinguish between different tomato leaf diseases.

10. Future Improvements

The current implementation can be extended in several ways:

- **Transfer Learning:** Implement pre-trained architectures like ResNet50, EfficientNet, or DenseNet for potentially higher accuracy.
- **More Training Data:** Collect additional images to improve model robustness.
- **Data Augmentation:** Apply more extensive augmentation techniques (rotation, zoom, flip, shear) to increase dataset diversity.
- **Hyperparameter Tuning:** Optimize learning rate, batch size, and network architecture.
- **Deployment:** Deploy the model as a mobile or web application using Flask, Streamlit, or TensorFlow Lite.

- Multi-class Extension: Extend the classifier to recognize multiple plant species and diseases.
 - Face Detection: Incorporate automated leaf detection and segmentation before classification.
-

11. Conclusion

This project successfully developed a CNN-based classifier for tomato leaf disease detection. The model was trained on a balanced dataset of **10,000 images across 10** categories, achieving a final training accuracy of approximately **94.7%** and a validation accuracy of **88.8%**.

The model demonstrated strong performance in distinguishing between different tomato leaf diseases, as evidenced by the high validation accuracy and excellent ROC-AUC scores. The use of dropout regularization helped prevent overfitting, and the simple CNN architecture proved effective for this classification task.

Overall, this project demonstrates the effectiveness of deep learning for agricultural applications, particularly for plant disease detection. With further improvements, such as transfer learning and data augmentation, the model could be enhanced for even better performance and real-world deployment.

12. References

1. TensorFlow Keras Documentation – CNN Layers.
2. Kaggle Dataset: Tomato Leaf Disease Dataset by kaustubhb999.
3. TensorFlow Documentation – ImageDataGenerator.
4. Chollet, F. (2018). Deep Learning with Python. Manning Publications.
5. Simonyan, K., & Zisserman, A. (2014). Very Deep Convolutional Networks for Large-Scale Image Recognition. arXiv:1409.1556.